

CliniScan: Lung-Abnormality Detection on Chest X-rays using AI.

Nilangi Mayuri

Table of Contents

1. Project Description	1
2. Dataset Used	1
3. Environment Setup	2
4. Data Exploration	3
5. Data Preprocessing 5.1. Image Loading 5.2. Ground Truth Extraction 5.3. Image Resizing & Downscaling 5.4. Density Map Generation 5.5. Tensor Conversion	4
6. Proposed System	6
7. Training Pipeline	7
8. Conclusion	8

1. Project Description

The CliniScan project focuses on developing an automated object detection system to identify and localize abnormalities in chest X-ray images.

The system is built using the YOLO object detection framework, which enables fast and accurate detection of abnormal regions in medical images.

Unlike simple image classification, this system performs:

- Detection of abnormal regions
- Localization using bounding boxes
- Deep learning-based feature extraction

The primary goal of this project is to design and implement a deep learning-based object detection model capable of accurately detecting and localizing abnormal regions in chest X-ray images using bounding box predictions.

2. Dataset Used

The project uses the **VinBigData Chest X-ray Abnormalities Detection Dataset**, obtained from Kaggle. The dataset was provided by Vingroup Big Data Institute. The complete dataset contains 15,000 chest X-ray images. A curated subset of 1000 images was used due to computational constraints.

For each image in the dataset:

- A DICOM (.dicom) image file is provided.
- A CSV annotation file contains bounding box coordinates for abnormalities.
- Each annotation entry includes:
 - image_id
 - class_name
 - class_id
 - x_min, y_min, x_max, y_max
 - rad_id

The annotation type is:

- Object Detection (Bounding Box Localization)
- Abnormality Classification.

3. Environment Setup

The project is implemented in Python using the Kaggle Notebook environment.

The key libraries required include:

- **Ultralytics (YOLO)** – Used to build, train, and evaluate the object detection model.
- **PyTorch (torch, torchvision)** – Backend deep learning framework used by YOLO for model training and GPU acceleration.
- **OpenCV (opencv-python) and Pillow (pillow)** – Used for loading, resizing, converting DICOM to PNG, and manipulating images.
- **NumPy and Pandas** – For numerical computations and handling annotation CSV files efficiently.
- **Scikit-learn** – Used for dataset splitting into train, validation, and test sets.
- **Matplotlib** – Used for visualizing images, bounding boxes, and training metrics.
- **TQDM** – Provides progress bars during dataset preprocessing and label generation.
- **PyYAML** – Used for reading and configuring the YOLO data.yaml file.

A structured dataset directory was created in YOLO format to ensure compatibility with the Ultralytics training pipeline.

4. Data Exploration

Before preprocessing, the dataset directory structure was explored to ensure correct configuration. The exploration involved:

4.1. Verifying Dataset Folder Paths

- Checking whether the DICOM dataset exists in the file system.
- Confirming the availability of the annotation CSV file.
- Verifying image ID consistency between images and CSV annotations.
- Counting total selected images (1000).
- Confirming annotation entries (4504 rows).

This ensured that image files and annotation entries were properly aligned before further processing.

4.2. Displaying a Sample Image

A sample DICOM image was loaded and converted to PNG format to visually confirm:

- Image resolution
- Image clarity
- Grayscale channel structure
- Correct conversion from DICOM

The image was resized to 512×512 pixels and displayed using Matplotlib to confirm proper loading.

4.3. Visualizing Ground Truth Annotations

The CSV file was inspected to extract bounding box coordinates for a selected image.

The following steps were performed:

- Extract x_min , y_min , x_max , y_max values
- Overlay bounding boxes on the image
- Verify correct alignment of boxes with abnormal regions
- Confirm coordinate scaling after resizing

This step ensured that:

- Bounding boxes correctly represent abnormal regions
- Annotation format is consistent
- No mismatch exists between image size and bounding box values

This validation confirms that the dataset is correctly structured before proceeding with preprocessing and YOLO format conversion.

5. Data Preprocessing

This stage involves converting raw DICOM chest X-ray images and CSV bounding box annotations into YOLO-ready deep learning inputs. The preprocessing pipeline includes the following key steps:

5.1. Image Loading

Images are loaded from DICOM format using medical image processing utilities and converted into PNG format. The images are read using OpenCV and converted from BGR to RGB format for visualization consistency.

Purpose:

- Ensures medical images are properly converted from DICOM to standard image format
- Maintains consistent grayscale/RGB format for training

5.2. Ground Truth Extraction

The dataset provides annotations in a CSV file containing bounding box coordinates.

Steps:

- Load CSV file using Pandas
- Filter annotations corresponding to selected image IDs
- Extract bounding box values (x_min, y_min, x_max, y_max)

Purpose:

- Converts structured CSV annotation data into usable Python arrays
- Provides exact abnormal region locations for object detection training

5.3. Image Resizing & Standardization

Original chest X-ray images vary in resolution and aspect ratio. For training efficiency and consistent model input, images are resized.

Resizing operations used:

(1) Resize all images to 512×512

This becomes the final default resolution used for training.

Purpose:

- Reduces GPU memory usage
- Speeds up training process

5.4. Bounding Box Conversion to YOLO Format

This is the most important preprocessing stage.

Original annotation format:

- x_min
- y_min
- x_max
- y_max

Converted to YOLO format:

class_id center_x center_y width height

Steps:

1. Compute center_x and center_y from bounding box corners
2. Compute width and height
3. Normalize all values between 0 and 1 (divide by image width and height)
4. Save each image's annotations into corresponding .txt file

Purpose:

- Converts dataset into YOLO-compatible format
- Ensures proper bounding box localization during inference

5.5. Dataset Structuring & Tensor Preparation

YOLO requires a specific folder structure.

Dataset directory was organized as:

dataset/

```

├── images/
│   ├── train/
│   ├── val/
│   └── test/
├── labels/
│   ├── train/
│   ├── val/
│   └── test/

```

After loading during training:

- Image Tensor $\rightarrow (C \times H \times W)$ normalized image
- Target Tensor $\rightarrow$ bounding box annotations

Purpose:

- Converts images and labels into PyTorch tensors internally
- Prepares input-output pairs for YOLO training
- Enables batch-based GPU training

6. Proposed System

1. Raw High-Resolution Chest X-ray Image

The system begins with the original chest X-ray image obtained from the VinBigData dataset. These images contain varying resolutions and abnormal patterns requiring standardization before training.

2. Preprocessing (Resize & Normalize)

The image is resized to 512×512 and normalized.
This ensures consistent input dimensions and stabilizes training.

3. YOLO Model (Training / Inference)

The preprocessed image is passed to the YOLO object detection architecture, which consists of:

- Backbone network for feature extraction
- Neck for feature aggregation
- Detection head for bounding box and class prediction

The model learns spatial features required to identify abnormal lung regions.

4. Bounding Box Prediction

The model outputs bounding boxes around detected abnormal regions.

Each prediction includes:

- Bounding box coordinates
- Confidence score
- Class prediction

5. Final Detection Output

The final output is:

- Chest X-ray image with localized abnormal regions
- Confidence-based filtering of predictions
- Detection metrics (mAP, precision, recall)

This output is used for evaluation and performance comparison.

7. Training Pipeline

Training script includes:

Features:

- Checkpoint saving after each epoch
- Auto-resume from last checkpoint
- Validation after each epoch
- Automatic loss tracking
- Metric evaluation (mAP, precision, recall)

Why YOLO-based training?

- YOLO performs real-time object detection
- Efficient single-stage detection architecture
- Suitable for medical localization tasks

Training Configuration:

- Image Size: 512×512
- Epochs: 25
- Optimizer: SGD / Adam
- Pretrained weights used for transfer learning

Evaluation Metrics:

- Box Loss
- Classification Loss
- mAP@0.5
- mAP@0.5:0.95
- Precision
- Recall

These metrics measure detection accuracy and localization quality.

8. Conclusion

The preprocessing stage of YOLO-based CliniScan abnormality detection project was completed successfully. All selected DICOM chest X-ray images and their corresponding CSV bounding box annotations were loaded, verified, and analysed to understand abnormal region distributions.

Each image was converted from DICOM to PNG, resized to 512×512 resolution, and normalized for training consistency. Bounding box annotations were accurately converted into YOLO format and saved in structured label files.

The dataset was then split into training, validation, and testing sets and organized into YOLO-compatible directories. These processed image–annotation pairs formed the foundation for training and evaluating the deep learning model.